

DAY-5 — Processes & Services

CyberNotes • Detailed Beginner Reference (Improved Edition)

Learning goal: Build a practical, working understanding of **processes & services**, connect the concept to real cybersecurity work, and practice it safely in a local lab.

Study rule: Understand a command or concept before copying it. Security tools are powerful because they interact with real systems — your practice environment should always be intentionally controlled and authorized.

1. Core Concepts

Core idea: Processes & Services is a building block for practical cybersecurity. This section explains what each term actually means, where you'll encounter it, and how it connects to the bigger picture.

Why it matters: Security work depends on understanding normal system and network behavior before you can recognize abnormal behavior. This topic connects directly to Linux, networking, web security, monitoring, and troubleshooting — the threads running through this whole 20-day series.

Vocabulary covered: processes, PID, ps, top, jobs, signals, systemd, services.

Reference	What to remember
ps aux	process list
top	live process view
pgrep name	find process IDs
kill PID	send signal
systemctl status X	service status
journalctl -u X	service logs

2. Concept Explanations

Each term below is explained in plain language, with a note on why it matters for security work. Read these before running any commands.

1. processes

A process is a running instance of a program, with its own memory, open files, and execution state.

Understanding what processes should normally be running on a system is the baseline you need before you can recognize something abnormal — like an unexpected process making outbound network connections.

2. PID

Every process gets a unique Process ID (PID) while it runs, which is how you target it precisely with commands like `kill`. PIDs are reused after a process ends, so a PID alone doesn't identify a process across time — you also need to know when it started.

3. ps

`ps aux` lists all currently running processes with details like owning user, CPU/memory usage, and the full command line that launched them. It's a snapshot in time — useful for a quick check or for piping into `grep` to find a specific process.

4. top

`top` (or the more modern `htop`) shows a live, continuously updating view of processes sorted by resource usage, which is useful for spotting something consuming abnormal CPU or memory in real time, such as a runaway or malicious process.

5. jobs

Jobs refers to processes you started from your own shell session, which can be run in the background (`&`), suspended (`Ctrl+Z`), and managed with `jobs`, `fg`, and `bg`. This matters when running long tasks like scans so they don't block your terminal.

6. signals

A signal is a message sent to a process to tell it to do something — `SIGTERM` (15) politely asks it to stop, `SIGKILL` (9) forces immediate termination, `SIGHUP` (1) often tells a service to reload its config. `kill PID` sends `SIGTERM` by default; `kill -9 PID` sends `SIGKILL` when a process won't respond.

7. systemd

`systemd` is the init system used by most modern Linux distributions — it's the first process to start (PID 1) and is responsible for starting, stopping, and supervising all other services on the machine. Nearly all service management on a modern distro goes through `systemctl`, which talks to `systemd`.

8. services

A service is a program `systemd` manages in the background, usually without a terminal attached — web servers, SSH daemons, and databases are all typically run as services. `systemctl status` tells you whether a service is running, when it started, and its recent log output, all in one command.

9. logs

Service logs record what a program did and when, and on `systemd`-based systems the default source is the journal, viewed with `journalctl -u`. Logs are often the first place you look when a service misbehaves or when investigating a potential security incident (this connects directly to Day 19).

10. resource checks

Resource checks — CPU, memory, disk, and open connections — establish a system's normal baseline so you can recognize abuse later, such as a cryptominer quietly consuming CPU or a compromised service opening unexpected connections. `top`, `ps aux --sort=-%cpu`, and `ss` are common starting points.

3. Safe Practical Lab

Step 1. Define the lab target and the exact question you are trying to answer.

Step 2. Run the smallest safe observation or command that can answer that question.

Step 3. Read the output carefully; record important fields, errors, timestamps, and assumptions.

Step 4. Change one variable at a time and compare the result.

Step 5. Write a short conclusion and clean up temporary files or services.

Example lab sequence:

```
ps
ps aux | head
systemctl --type=service --state=running 2>/dev/null | head
```

4. Common Mistakes & Troubleshooting

- Running a command without checking its target, current directory, or privileges.
- Confusing a tool's output with proof of a vulnerability or compromise — output is evidence, not a conclusion.
- Ignoring error messages instead of reading the exact reason for failure.
- Changing several settings at once, making it hard to tell what actually fixed or broke the lab.
- Saving real credentials, tokens, or personal data in practice notes or public repositories.
- Testing a public target just because it's reachable — reachability is never the same as authorization.

Troubleshooting checklist:

- What system am I on?
- What exact command/request did I run?
- What did I expect?
- What happened instead?
- What does the error message say?
- Can I reproduce it in a clean local lab?
- What changed since the last successful attempt?

5. Deep-Dive Questions

Answer these in your own words after finishing the lab — they're designed to check real understanding, not memorization.

Q1. What problem does processes solve, in your own words?

Notes: _____

Q2. What is the difference between normal behavior and a suspicious observation in this topic?

Notes: _____

Q3. Which command, log, packet, file, or response would you inspect first, and why?

Notes: _____

Q4. What could cause a misleading result or false positive here?

Notes: _____

Q5. How would you document your observation so another learner could reproduce it safely?

Notes: _____

Q6. Which security control reduces the risk associated with this topic?

Notes: _____

6. Revision Checklist

- I can explain processes & services in my own words.
- I can identify the main terms without looking at the notes.
- I can perform the example in a controlled lab.
- I can interpret the output instead of only copying it.
- I know at least one defensive or troubleshooting use for the concept.
- I can document the result clearly and safely.

7. Safety & Scope

Use these commands and tools only on systems you own, a local virtual machine, a CTF, an intentionally vulnerable application, or a target for which you have explicit written permission.

Keep a written scope: target, allowed actions, time window, and stop conditions. Never scan or test random public systems.

When practicing, save your notes and outputs so you can explain what you observed rather than blindly running commands.

Next step: After Day-5, review your notes and connect this topic to the next subject in the series. The goal is not to memorize commands — it's to understand systems well enough to reason about security.

8. Quick Reference & Personal Notes

Topic: Day-5 — Processes & Services

Three things I learned:

1. _____
1. _____
1. _____

Commands / concepts to memorize:

- `ps aux` — process list
- `top` — live process view
- `pgrep name` — find process IDs
- `kill PID` — send signal
- `systemctl status X` — service status
- `journalctl -u X` — service logs

My lab result

What I tested: _____

What I observed: _____

What I would investigate next: _____

Final reminder: A strong cybersecurity learner can explain both the offensive concept and the defensive reason for learning it. Keep practice authorized, reproducible, and documented.